Practical Exam Solutions

Slip 5 - Java

```
Ubuntu Steps:
1. Create a dummy file: echo "Hello World" > test.txt
2. gedit Slip5_1.java → javac Slip5_1.java → java Slip5_1
```

Write a program to accept a text file from user and display the contents of a file in reverse order and change its case.

```
import java.io.*;
import java.util.Scanner;

class Slip5_1 {
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String name = sc.nextLine();

        FileReader fr = new FileReader(name);
        StringBuilder sb = new StringBuilder();
        int ch;

        while ((ch = fr.read()) != -1) {
            sb.append((char) ch);
        }
        fr.close();

        sb.reverse(); // Reverse the string built from the file

        for (int i = 0; i < sb.length(); i++) {
            char c = sb.charAt(i);
            if (Character.isUpperCase(c)) {
                System.out.print(Character.toLowerCase(c));
            } else {
                System.out.print(Character.toUpperCase(c));
            }
        }
        System.out.println();
    }
}
```

Slip 5 - Web Technology

```
Ubuntu Steps:
1. Ensure you have dummy images (img1.jpg, img2.jpg, etc.) in the same folder.
2. gedit Slip5_2.html → Open in browser: firefox Slip5_2.html
```

Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

```
<!DOCTYPE html>
<html>
<head>
<style>
  .gallery {
      display: grid;
      grid-template-columns: repeat(3, 1fr);
      gap: 15px;
  }
  .gallery img {
      width: 100%;
      height: 200px;
      object-fit: cover;
      transition: transform 0.3s ease;
  }
  .gallery img:hover {
      transform: scale(1.1);
  }
</style>
</head>
<body>
  <h2>CSS Grid Image Gallery</h2>
  <div class="gallery">
    <img src="img1.jpg" alt="Gallery Image 1">
    <img src="img2.jpg" alt="Gallery Image 2">
    <img src="img3.jpg" alt="Gallery Image 3">
  </div>
</body>
</html>
```

Slip 6 - Java

Ubuntu Steps:
1. gedit Slip6_1.java → javac Slip6_1.java → java Slip6_1

Write a program to define a class Account (custname, accno). Subclass SavingAccount (savingbal, minbal). Subclass AccountDetail (depositamt, withdrawalamt). Display details.

```
class Account {
    String custname;
    int accno;
    Account() {}
    Account(String c, int a) { custname = c; accno = a; }
}

class SavingAccount extends Account {
    double savingbal, minbal;
    SavingAccount(String c, int a, double s, double m) {
        super(c, a);
        savingbal = s;
        minbal = m;
    }
}

class AccountDetail extends SavingAccount {
    double depositamt, withdrawalamt;
    AccountDetail(String c, int a, double s, double m, double d, double w) {
        super(c, a, s, m);
        depositamt = d;
        withdrawalamt = w;
    }
    void display() {
        System.out.println("Customer Name: " + custname);
        System.out.println("Account Number: " + accno);
        System.out.println("Savings Balance: " + savingbal);
        System.out.println("Minimum Balance: " + minbal);
        System.out.println("Deposit Amount: " + depositamt);
        System.out.println("Withdrawal Amount: " + withdrawalamt);
    }
}

class Slip6_1 {
    public static void main(String[] args) {
        AccountDetail obj = new AccountDetail("Ramesh", 101, 15000, 1000, 5000,
2000);
        obj.display();
    }
}
```

Slip 6 - Web Technology

Ubuntu Steps:
1. gedit Slip6_2.html → firefox Slip6_2.html

Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

```
<!DOCTYPE html>
<html>
<head>
<style>
  .my-button {
      padding: 10px 20px;
      background-color: lightblue;
      border: 1px solid blue;
      cursor: pointer;
  }
  .my-link {
      padding: 10px;
      color: blue;
      text-decoration: none;
      display: inline-block;
  }

  /* Hover effects */
  .my-button:hover {
      background-color: darkblue;
      color: white;
  }
  .my-link:hover {
      background-color: yellow;
      text-decoration: underline;
  }
</style>
</head>
<body>
  <h3>Hover Effects Demo</h3>
  <button class="my-button">Hover Over Me!</button>
  <br><br>
  <a href="#" class="my-link">Hover over this link!</a>
</body>
</html>
```

Slip 7 - Java

Ubuntu Steps:
1. gedit Slip7_1.java → javac Slip7_1.java → java Slip7_1

Class Driver (license_no, name, address, age). Initialize values through constructor. If age < 18 generate user-defined exception "Age is below 18 years".

```
class AgeException extends Exception {
    AgeException(String msg) {
        super(msg);
    }
}

class Driver {
    String license_no, name, address;
    int age;

    Driver(String l, String n, String a, int age) {
        license_no = l;
        name = n;
        address = a;
        this.age = age;
    }

    void display() {
        try {
            if (age < 18) {
                throw new AgeException("Age is below 18 years");
            } else {
                System.out.println("Driver Name: " + name);
                System.out.println("License: " + license_no);
                System.out.println("Address: " + address);
            }
        } catch (AgeException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

class Slip7_1 {
    public static void main(String[] args) {
        Driver d1 = new Driver("DL-1234", "Sachin", "Mumbai", 16);
        d1.display(); // Will throw exception

        System.out.println("-----------------");

        Driver d2 = new Driver("DL-9999", "Virat", "Delhi", 25);
        d2.display(); // Will display details
    }
```

Slip 7 - Web Technology

Ubuntu Steps:
1. gedit Slip7_2.html → firefox Slip7_2.html

Create an unordered list of Programming Language names and apply different styles to the first and last list items. (Use :first-child and :last-child)

```
<!DOCTYPE html>
<html>
<head>
<style>
  ul {
      font-size: 18px;
  }
  li:first-child {
      color: red;
      font-weight: bold;
      font-size: 22px;
  }
  li:last-child {
      color: green;
      text-decoration: underline;
      font-style: italic;
  }
</style>
</head>
<body>
  <h3>Programming Languages</h3>
  <ul>
    <li>Java</li>
    <li>Python</li>
    <li>C++</li>
    <li>JavaScript</li>
    <li>PHP</li>
  </ul>
</body>
</html>
```

Slip 8 - Java

Ubuntu Steps:
1. gedit Slip8_1.java → javac Slip8_1.java → java Slip8_1

Abstract class Shape with printArea(). Subclasses Rectangle, Triangle, Circle override printArea(). Use method overriding.

```
abstract class Shape {
    int a = 10, b = 5;
    abstract void printArea();
}

class Rectangle extends Shape {
    void printArea() {
        System.out.println("Area of Rectangle = " + (a * b));
    }
}

class Triangle extends Shape {
    void printArea() {
        System.out.println("Area of Triangle = " + (0.5 * a * b));
    }
}

class Circle extends Shape {
    void printArea() {
        // Using 'a' as radius
        System.out.println("Area of Circle = " + (3.14 * a * a));
    }
}

class Slip8_1 {
    public static void main(String[] args) {
        Shape s; // Reference of superclass

        s = new Rectangle();
        s.printArea();

        s = new Triangle();
        s.printArea();

        s = new Circle();
        s.printArea();
    }
}
```

Slip 8 - Web Technology

```
Ubuntu Steps:
1. Ensure two dummy images (1.jpg, 2.jpg) exist in the folder.
2. gedit Slip8_2.html → firefox Slip8_2.html
```

Create a webpage using JavaScript DOM manipulation. Click button to replace displayed image with another image.

```
<!DOCTYPE html>
<html>
<head>
    <title>Change Image via DOM</title>
</head>
<body>
  <h3>JavaScript Image Replacement</h3>

  <!-- Image ID is used to target it via JavaScript -->
  <img id="targetImg" src="1.jpg" width="300" height="200" alt="First Image">
  <br><br>

  <button onclick="changeMyImage()">Click to Change Image</button>

  <script>
    function changeMyImage() {
        // Targets the image and updates its source attribute
        document.getElementById('targetImg').src = "2.jpg";
    }
  </script>
</body>
</html>
```

Slip 9 - Java

Ubuntu Steps:
1. gedit Slip9_1.java → javac Slip9_1.java → java Slip9_1

Class Employee (id, name, salary). Store information of 'n' employees. Display name of employee having maximum salary (Use array of objects).

```
import java.util.Scanner;

class Employee {
    int id;
    String name;
    double salary;

    Employee(int i, String n, double s) {
        id = i;
        name = n;
        salary = s;
    }
}

class Slip9_1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of employees: ");
        int n = sc.nextInt();

        // Array of Objects
        Employee[] emp = new Employee[n];

        for(int i = 0; i < n; i++) {
            System.out.print("Enter ID, Name, Salary for Emp " + (i+1) + ": ");
            int id = sc.nextInt();
            String name = sc.next();
            double salary = sc.nextDouble();
            emp[i] = new Employee(id, name, salary);
        }

        // Finding max salary
        int maxIndex = 0;
        for(int i = 1; i < n; i++) {
            if(emp[i].salary > emp[maxIndex].salary) {
                maxIndex = i;
            }
        }

        System.out.println("\nEmployee with Maximum Salary:");
        System.out.println("Name: " + emp[maxIndex].name + " | Salary: " +
emp[maxIndex].salary);
    }
```

Slip 9 - Web Technology

Ubuntu Steps:
1. gedit Slip9_2.html → firefox Slip9_2.html

JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division.

```
<!DOCTYPE html>
<html>
<body>
  <h3>Arrow Functions Demo</h3>
  <script>
    // Arrow functions defined with const
    const add = (a, b) => a + b;
    const sub = (a, b) => a - b;
    const mul = (a, b) => a * b;
    const div = (a, b) => a / b;

    let x = 20;
    let y = 5;

    document.write("Numbers: " + x + " and " + y + "<br><br>");
    document.write("Addition = " + add(x, y) + "<br>");
    document.write("Subtraction = " + sub(x, y) + "<br>");
    document.write("Multiplication = " + mul(x, y) + "<br>");
    document.write("Division = " + div(x, y));
  </script>
</body>
</html>
```

Slip 10 - Java

Ubuntu Steps:
1. echo "this is lowercase text" > abc.txt
2. gedit Slip10_1.java → javac Slip10_1.java → java Slip10_1

Write a program to read the contents of "abc.txt" file. Display the contents of file in uppercase as output.

```
import java.io.*;

class Slip10_1 {
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("abc.txt");
            int ch;

            System.out.println("File Contents in Uppercase:");
            // read() returns -1 when end of file is reached
            while ((ch = fr.read()) != -1) {
                System.out.print(Character.toUpperCase((char) ch));
            }

            fr.close();
            System.out.println(); // Adding a newline at the end

        } catch (Exception e) {
            System.out.println("Error reading file: " + e);
        }
    }
}
```

Slip 10 - Web Technology

Ubuntu Steps:
1. gedit Slip10_2.html → firefox Slip10_2.html

JavaScript program to format a student report using template literals (`backticks` and ${variable}).

```
<!DOCTYPE html>
<html>
<body>
  <h3>Template Literals Output:</h3>
  <script>
    // Storing data in variables
    const studentName = "Sneha";
    const rollNo = 105;
    const marks = 92;

    // Using template literals for multi-line formatted string
    let reportCard = `
      <div style="border: 2px solid black; padding: 10px; width: 300px;">
          <h4 style="margin:0;">Student Report Card</h4>
          <hr>
          <p><b>Name:</b> ${studentName}</p>
          <p><b>Roll Number:</b> ${rollNo}</p>
          <p><b>Total Marks:</b> ${marks}%</p>
      </div>
    `;

    // Displaying the result
    document.write(reportCard);
  </script>
</body>
</html>
```

Slip 11 - Java

Ubuntu Steps:
1. gedit Slip11_1.java → javac Slip11_1.java → java Slip11_1

Super class Vehicle (Company, price). Derive LightMotorVehicle (mileage) and HeavyMotorVehicle (capacity). Accept 'n' vehicles, ask user about type first.

```
import java.util.Scanner;

class Vehicle {
    String company;
    double price;
    Vehicle(String c, double p) { company = c; price = p; }
}

class LightVehicle extends Vehicle {
    double mileage;
    LightVehicle(String c, double p, double m) {
        super(c, p);
        mileage = m;
    }
    void show() { System.out.println(company + " | Rs." + price + " | " + mileage
+ " kmpl"); }
}

class HeavyVehicle extends Vehicle {
    double capacity;
    HeavyVehicle(String c, double p, double cap) {
        super(c, p);
        capacity = cap;
    }
    void show() { System.out.println(company + " | Rs." + price + " | " + capacity
+ " tons"); }
}

class Slip11_1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vehicles: ");
        int n = sc.nextInt();

        for (int i = 0; i < n; i++) {
            System.out.print("\nEnter Type (1 for Light, 2 for Heavy): ");
            int type = sc.nextInt();
            System.out.print("Enter Company & Price: ");
            String comp = sc.next();
            double price = sc.nextDouble();

            if (type == 1) {
```

Slip 11 - Web Technology

Ubuntu Steps:
1. gedit Slip11_2.html → firefox Slip11_2.html

JavaScript program using an arrow function to calculate total and average marks of 3 subjects. Display using template literals.

```
<!DOCTYPE html>
<html>
<body>
  <script>
    // Arrow functions for logic
    const calcTotal = (s1, s2, s3) => s1 + s2 + s3;
    const calcAvg = (total) => total / 3;

    // Subject marks
    let mark1 = 75, mark2 = 80, mark3 = 90;

    let totalMarks = calcTotal(mark1, mark2, mark3);
    let avgMarks = calcAvg(totalMarks);

    // Output via Template Literals
    document.write(`
      <h3>Examination Results</h3>
      <p>Subject 1: ${mark1}</p>
      <p>Subject 2: ${mark2}</p>
      <p>Subject 3: ${mark3}</p>
      <hr>
      <p><b>Total Marks:</b> ${totalMarks}</p>
      <p><b>Average Marks:</b> ${avgMarks.toFixed(2)}</p>
    `);
  </script>
</body>
</html>
```

Slip 13 - Java

Ubuntu Steps:
1. gedit Slip13_1.java → javac Slip13_1.java → java Slip13_1

Define a "Clock" class: accept H, M, S. Check validity. Set time to AM/PM mode.

```
class Clock {
    int h, m, s;

    Clock(int h, int m, int s) {
        this.h = h;
        this.m = m;
        this.s = s;
    }

    boolean isValid() {
        return (h >= 0 && h < 24) && (m >= 0 && m < 60) && (s >= 0 && s < 60);
    }

    void display() {
        if (!isValid()) {
            System.out.println("Invalid Time Entered.");
            return;
        }

        String ampm = (h < 12) ? "AM" : "PM";

        // Convert to 12 hour format
        int displayHour = h % 12;
        if (displayHour == 0) {
            displayHour = 12; // Midnight or Noon
        }

        System.out.println("Standard Time: " + displayHour + ":" + m + ":" + s + "
" + ampm);
    }
}

class Slip13_1 {
    public static void main(String[] args) {
        System.out.println("Testing Clock logic:");
        Clock c1 = new Clock(14, 30, 45);
        c1.display(); // Should print 2:30:45 PM

        Clock c2 = new Clock(25, 10, 10);
        c2.display(); // Should print Invalid
    }
}
```

Slip 13 - Web Technology

Ubuntu Steps:
1. gedit Slip13_2.html → firefox Slip13_2.html

JavaScript program to swap two numbers without using a third variable using array destructuring.

```
<!DOCTYPE html>
<html>
<body>
  <h3>Swap via Array Destructuring</h3>
  <script>
    let x = 100;
    let y = 500;

    document.write(`<p>Before Swapping: x = ${x}, y = ${y}</p>`);

    // Array Destructuring Swap Magic
    [x, y] = [y, x];

    document.write(`<p>After Swapping: x = ${x}, y = ${y}</p>`);
  </script>
</body>
</html>
```

Slip 19 - Java

```
Ubuntu Steps:
1. echo -e "Line one\nLine two text" > sample.txt
2. gedit Slip19_1.java → javac Slip19_1.java → java Slip19_1
```

Accept file name and count number of characters, words, and lines present in the file.

```
import java.io.*;
import java.util.Scanner;

class Slip19_1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String fileName = sc.nextLine();

        try {
            BufferedReader br = new BufferedReader(new FileReader(fileName));
            int chars = 0, words = 0, lines = 0;
            String lineData;

            // Read file line by line
            while ((lineData = br.readLine()) != null) {
                lines++;
                chars += lineData.length();

                // Split line into words using space delimiter
                String[] wordList = lineData.split("\\s+");
                words += wordList.length;
            }
            br.close();

            System.out.println("Total Lines: " + lines);
            System.out.println("Total Words: " + words);
            System.out.println("Total Characters: " + chars);

        } catch (Exception e) {
            System.out.println("File Error: " + e.getMessage());
        }
    }
}
```

Slip 19 - Web Technology

Ubuntu Steps:
1. gedit Slip19_2.html → firefox Slip19_2.html

JavaScript program using async/await to simulate user login. Success message if correct, handle error via try-catch if incorrect.

```
<!DOCTYPE html>
<html>
<body>
  <h3>Async / Await Login Simulation</h3>
  <script>
    // Async function returning a Promise based on condition
    async function performLogin(username, password) {
        if (username === "admin" && password === "pass123") {
            return "Login Successful! Welcome Admin.";
        } else {
            throw "Login Failed! Invalid Username or Password.";
        }
    }

    // Caller function with Try-Catch
    async function testLogin() {
        try {
            // Testing with valid credentials
            let result = await performLogin("admin", "pass123");
            document.write(`<p style="color:green">${result}</p>`);

            // Testing with invalid credentials (will jump to catch)
            await performLogin("user", "wrong");
        }
        catch (errorMsg) {
            document.write(`<p style="color:red">${errorMsg}</p>`);
        }
    }

    testLogin();
  </script>
</body>
</html>
```

Slip 21 - Java

Ubuntu Steps:
1. gedit Slip21_1.java → javac Slip21_1.java → java Slip21_1

Parent class College (cno, cname, caddr) and derived class Department (dno, dname). Display details.

```
class College {
    int cno;
    String cname, caddr;

    College(int no, String name, String addr) {
        cno = no;
        cname = name;
        caddr = addr;
    }
}

class Department extends College {
    int dno;
    String dname;

    Department(int c_no, String c_name, String c_addr, int d_no, String d_name) {
        super(c_no, c_name, c_addr); // Call parent constructor
        dno = d_no;
        dname = d_name;
    }

    void displayDetails() {
        System.out.println("--- College Details ---");
        System.out.println("ID: " + cno + " | Name: " + cname + " | Addr: " +
caddr);
        System.out.println("--- Department Details ---");
        System.out.println("Dept No: " + dno + " | Dept Name: " + dname);
    }
}

class Slip21_1 {
    public static void main(String[] args) {
        Department d = new Department(1, "Pune University", "Pune", 10, "Computer
Science");
        d.displayDetails();
    }
}
```

Slip 21 - Web Technology (Node.js)

Ubuntu Steps:
1. gedit Slip21_2.js → node Slip21_2.js

Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

```
const path = require("path");

// A sample path string
let samplePath = "/home/student/desktop/project/index.html";

console.log("--- Path Module Operations ---");

// 1. Extract Directory Name
console.log("Directory:", path.dirname(samplePath));

// 2. Extract Base (File) Name
console.log("File Name:", path.basename(samplePath));

// 3. Extract Extension
console.log("Extension:", path.extname(samplePath));

// 4. Joining Paths
let joinedPath = path.join("/home", "student", "images", "pic.jpg");
console.log("Joined Path:", joinedPath);

// 5. Resolving absolute path
let resolvedPath = path.resolve("script.js");
console.log("Resolved Path:", resolvedPath);
```

Slip 22 - Java

Ubuntu Steps:
1. gedit Slip22_1.java → javac Slip22_1.java → java Slip22_1

Interface, class Product (id, name, cost, qty). Create objects, display contents and display the object count.

```
interface Displayable {
    void display();
}

class Product implements Displayable {
    int id, qty;
    String name;
    double cost;
    static int objectCount = 0; // Static variable to track objects

    Product() {
        objectCount++;
    }

    Product(int i, String n, double c, int q) {
        id = i;
        name = n;
        cost = c;
        qty = q;
        objectCount++;
    }

    public void display() {
        System.out.println("ID:" + id + " | " + name + " | Rs." + cost + " | Qty:"
+ qty);
    }
}

class Slip22_1 {
    public static void main(String[] args) {
        System.out.println("Product Details:");
        Product p1 = new Product(101, "Laptop", 45000, 1);
        p1.display();

        Product p2 = new Product(102, "Mouse", 500, 3);
        p2.display();

        Product p3 = new Product(); // Default constructor

        System.out.println("\nTotal Objects Created = " + Product.objectCount);
    }
}
```

Slip 22 - Web Technology (Node.js)

Ubuntu Steps:
1. gedit Slip22_2.js → node Slip22_2.js
(This script will create and edit 'async_test.txt' automatically)

Node.js program using the fs module to perform file operations asynchronously.

```
const fs = require("fs");

console.log("Starting Async File Operations...");

// 1. Write to file (Overwrites if exists)
fs.writeFile("async_test.txt", "Hello from Node.js! ", (err) => {
    if (err) throw err;
    console.log("File Write Completed.");

    // 2. Append to file (Happens only after write completes)
    fs.appendFile("async_test.txt", "This text is appended.", (err) => {
        if (err) throw err;
        console.log("File Append Completed.");

        // 3. Read from file (Happens only after append completes)
        fs.readFile("async_test.txt", "utf8", (err, data) => {
            if (err) throw err;
            console.log("\n--- Final File Content ---");
            console.log(data);
        });
    });
});
```

Slip 23 - Java

Ubuntu Steps:
1. gedit Slip23_1.java → javac Slip23_1.java → java Slip23_1

Multilevel inheritance: Continent -> Country -> State. Display place, State, Country and Continent.

```
class Continent {
    String contName;
    Continent(String c) { contName = c; }
}

class Country extends Continent {
    String ctryName;
    Country(String c, String y) {
        super(c);
        ctryName = y;
    }
}

class State extends Country {
    String stateName, placeName;

    State(String cont, String ctry, String state, String place) {
        super(cont, ctry);
        stateName = state;
        placeName = place;
    }

    void displayHierarchy() {
        System.out.println("Place: " + placeName);
        System.out.println("State: " + stateName);
        System.out.println("Country: " + ctryName);
        System.out.println("Continent: " + contName);
    }
}

class Slip23_1 {
    public static void main(String[] args) {
        State s = new State("Asia", "India", "Maharashtra", "Nashik");
        s.displayHierarchy();
    }
}
```

Slip 23 - Web Technology (Node.js)

Ubuntu Steps:
1. gedit Slip23_2.js → node Slip23_2.js

Node.js program to create, write, read and append data to a file using fs module.

```
const fs = require("fs");
let filename = "slip23_data.txt";

// Write / Create
fs.writeFile(filename, "Initial Text.\n", (err) => {
    if (err) throw err;
    console.log("1. File Created & Written.");

    // Append
    fs.appendFile(filename, "Appended Text.", (err) => {
        if (err) throw err;
        console.log("2. Data Appended.");

        // Read
        fs.readFile(filename, "utf8", (err, fileData) => {
            if (err) throw err;
            console.log("\n3. Reading File Data:");
            console.log(fileData);
        });
    });
});
```

Slip 24 - Java

Ubuntu Steps:
1. gedit Slip24_1.java → javac Slip24_1.java → java Slip24_1

Abstract class Shape with methods area & volume. Derive class Cylinder (radius, height). Calculate area and volume.

```
abstract class Shape {
    abstract void area();
    abstract void volume();
}

class Cylinder extends Shape {
    double radius, height;

    Cylinder(double r, double h) {
        radius = r;
        height = h;
    }

    void area() {
        // Area of Cylinder = 2πr(r + h)
        double a = 2 * Math.PI * radius * (radius + height);
        System.out.println("Surface Area of Cylinder = " + a);
    }

    void volume() {
        // Volume of Cylinder = πr²h
        double v = Math.PI * radius * radius * height;
        System.out.println("Volume of Cylinder = " + v);
    }
}

class Slip24_1 {
    public static void main(String[] args) {
        Cylinder cyl = new Cylinder(5.0, 10.0);
        cyl.area();
        cyl.volume();
    }
}
```

Slip 24 - Web Technology (Node.js)

Ubuntu Steps:
1. echo "Execution Data" > sync_data.txt
2. gedit Slip24_2.js → node Slip24_2.js

Node.js program to demonstrate synchronous and asynchronous file operations and compare execution.

```
const fs = require('fs');

console.log("Program Started.");

// 1. Synchronous Operation (Blocks the main thread)
console.log("\nExecuting Sync Read:");
let syncData = fs.readFileSync('sync_data.txt', 'utf8');
console.log("-> Sync Output:", syncData.trim());
console.log("Sync Read Finished.");

// 2. Asynchronous Operation (Non-blocking)
console.log("\nExecuting Async Read:");
fs.readFile('sync_data.txt', 'utf8', (err, asyncData) => {
    if (err) throw err;
    console.log("-> Async Output:", asyncData.trim());
});

// This will print BEFORE the async read completes
console.log("Program Ended. (Notice how Async finishes last)");
```

Slip 26 - Java

Ubuntu Steps:
1. gedit Slip26_1.java → javac Slip26_1.java → java Slip26_1

Class Student. If attendance percentage is less than 75%, throw exception "Student is Not Eligible for Exam".

```
class AttendanceException extends Exception {
    AttendanceException(String msg) { super(msg); }
}

class Student {
    String name;
    int roll, totalLectures, attendedLectures;

    Student(String n, int r, int t, int a) {
        name = n; roll = r; totalLectures = t; attendedLectures = a;
    }

    void checkEligibility() {
        try {
            double percent = (attendedLectures * 100.0) / totalLectures;

            if (percent < 75.0) {
                throw new AttendanceException("Student is Not Eligible for Exam");
            } else {
                System.out.println("Student: " + name + " (Roll: " + roll + ")");
                System.out.println("Attendance: " + percent + "% -> Eligible.");
            }
        } catch (AttendanceException e) {
            System.out.println("Error for " + name + ": " + e.getMessage());
        }
    }
}

class Slip26_1 {
    public static void main(String[] args) {
        new Student("Arun", 10, 100, 80).checkEligibility(); // Eligible
        new Student("Bob", 12, 100, 60).checkEligibility();  // Throws exception
    }
}
```

Slip 26 - Web Technology (Node.js)

Ubuntu Steps:
1. echo "file content" > original.txt
2. gedit Slip26_2.js → node Slip26_2.js

Node.js program using fs module to copy, rename and delete files asynchronously.

```
const fs = require("fs");

// 1. Copy
fs.copyFile("original.txt", "copied.txt", (err) => {
    if (err) return console.log("Copy Error:", err);
    console.log("Success: File copied.");

    // 2. Rename (happens after copy)
    fs.rename("copied.txt", "renamed.txt", (err) => {
        if (err) return console.log("Rename Error:", err);
        console.log("Success: File renamed.");

        // 3. Delete (happens after rename)
        fs.unlink("renamed.txt", (err) => {
            if (err) return console.log("Delete Error:", err);
            console.log("Success: File deleted from system.");
        });
    });
});
```

Slip 29 - Java

Ubuntu Steps:
1. gedit Slip29_1.java → javac Slip29_1.java → java Slip29_1

Accept number. If 0, throw exception. Otherwise check prime. (Use static keyword)

```
import java.util.Scanner;

class ZeroException extends Exception {
    ZeroException(String m) { super(m); }
}

class Slip29_1 {
    // Static method as requested
    static void checkPrime(int n) {
        try {
            if (n == 0) throw new ZeroException("Number is 0");

            boolean isPrime = true;
            if (n < 2) isPrime = false; // 1 and negatives are not prime

            for (int i = 2; i <= n/2; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }

            System.out.println(n + " is " + (isPrime ? "Prime" : "Not Prime"));

        } catch (ZeroException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = sc.nextInt();
        checkPrime(num); // Calling static method directly
    }
}
```

Slip 29 - Web Technology (Node.js)

Ubuntu Steps:
1. gedit Slip29_2.js → node Slip29_2.js

Node.js program to demonstrate file statistics using fs.stat(), including size, type, and dates.

```
const fs = require("fs");

// Checking stats of the script file itself
fs.stat("Slip29_2.js", (err, stats) => {
    if (err) {
        return console.log("Error reading stats:", err.message);
    }

    console.log("--- File Statistics ---");
    console.log("Size in bytes:", stats.size);

    if (stats.isFile()) console.log("Type: It is a File.");
    if (stats.isDirectory()) console.log("Type: It is a Directory.");

    console.log("Created On:", stats.birthtime);
    console.log("Last Accessed:", stats.atime);
    console.log("Last Modified:", stats.mtime);
});
```

Slip 12 - Java

Ubuntu Steps:

```
1. mkdir -p game && gedit game/Indoor.java game/Outdoor.java Slip12_1.java
2. javac game/*.java Slip12_1.java -> java Slip12_1
```

Write a package game with 2 classes Indoor & Outdoor. Use function display() to list players.

```
// Indoor.java (Inside game/ directory)
package game;
public class Indoor {
    String game;
    public Indoor() { game = "Chess"; }
    public Indoor(String g) { game = g; }
    public void display() {
        System.out.println("Indoor Game: " + game + " | Players: 2");
    }
}

// Outdoor.java (Inside game/ directory)
package game;
public class Outdoor {
    String game;
    public Outdoor() { game = "Cricket"; }
    public Outdoor(String g) { game = g; }
    public void display() {
        System.out.println("Outdoor Game: " + game + " | Players: 11");
    }
}

// Slip12_1.java (Main file in Seat_NO/)
import game.*;
class Slip12_1 {
    public static void main(String args[]) {
        Indoor i = new Indoor("Carrom");
        Outdoor o = new Outdoor("Football");
        i.display();
        o.display();
    }
}
```

Slip 12 - Web Technology

Ubuntu Steps:

```
1. gedit Slip12_2.html -> Open in browser: firefox Slip12_2.html
```

JavaScript program using arrow function to calculate product total and print via template literals.

```
<!DOCTYPE html>
<html>
<head>
    <title>Slip 12 - Product Bill</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 20px; }
        .bill-card { border: 1px solid #ccc; padding: 15px; width: 280px; }
    </style>
</head>
<body>
    <h3>Product Invoice</h3>
    <div class="bill-card">
    <script>
        // Arrow function for total calculation
        const total = (price, qty) => price * qty;
        const product = "Pen";
        const price = 20;
        const qty = 5;

        // String interpolation with template literals
        document.write(`
            <p><strong>Product:</strong> ${product}</p>
            <p><strong>Price:</strong> Rs. ${price}</p>
            <p><strong>Quantity:</strong> ${qty}</p>
            <hr>
            <p><strong>Total Bill:</strong> Rs. ${total(price, qty)}</p>
        `);
    </script>
    </div>
</body>
</html>
```